The Five Sticking Points, Taught Slowly

A clarification supplement to the Interpretation Playbook — plus everything the playbook skipped from your three session guides

Machine Learning for Managers · Prepared for Kishore Kunal (38BM21), MDI Gurgaon

WHAT'S INSIDE (ONLY WHAT YOU ASKED FOR)

1. Probability vs odds vs log-odds — three languages for one belief, when each is needed, and exactly how the Q1 calculation (compute a probability) differs from Q2 (interpret a coefficient).
2. The confusion matrix, actually drawn — a bank example filled in cell by cell, with what each cell costs.
3 & 4. Why one model is 'better' — the same bank data with a second model and rupee values attached, so the two-model comparison stops being 'just numbers'. Then the same routine run on spam, where the verdict flips.
5. Thresholds: not hit-and-trial — the sorted-scores sweep that makes it a real calculation, the cost method, and Youden's J finally made clear (including what that 0.55 actually was).
6. Gap-check — eight interpretation topics from your Linear Regression, Decision Tree and Logistic Regression guides that the playbook didn't cover, each compressed to its exam essence.

1. Probability, odds, log-odds — three languages for one belief

Your confusion is the right confusion to have — these are three different NUMBERS describing the SAME belief, and exams deliberately switch languages mid-question. Here is the whole system in one picture:

Language	The question it answers	Range	Example: 'rain is very likely'
Probability (p)	Out of everything that could happen, what SHARE is Yes?	0 to 1	p = 0.80 — 'an 80% chance'
Odds	How many Yeses PER No?	0 to ∞	odds = 0.80 ÷ 0.20 = 4 — '4-to-1: four rainy days for every dry one'
Log-odds (z)	The odds, stretched onto a straight-line scale	−∞ to +∞	ln(4) ≈ +1.39 — a raw score, meaningless to a manager until translated back

The conversion ladder (memorise the four moves)

From → To	How	Worked
Probability → Odds	odds = p ÷ (1 − p)	p = 0.8 → 0.8÷0.2 = 4
Odds → Probability	p = odds ÷ (1 + odds)	odds = 4 → 4÷5 = 0.8
Log-odds → Odds	odds = e^z (just un-log it)	z = −0.4 → $e^{-0.4} \approx 0.67$
Log-odds → Probability	p = $e^z \div (1 + e^z)$ — the sigmoid	z = −0.4 → 0.67÷1.67 ≈ 0.40

WHY THREE LANGUAGES EXIST AT ALL

Probability is the language of DECISIONS (it's what you compare with a threshold), but it is trapped between 0 and 1, so no straight line can fit it. Odds stretch it to 'zero to infinity'; taking the log stretches it the rest of the way to 'minus infinity to plus infinity' — and NOW a straight line fits. So logistic regression does its internal work in log-odds (that's your z), while you, the manager, always translate the final answer back to a probability. The model thinks in log-odds; the boardroom hears probability; odds are the bridge between them.

When does the exam need which?

The question says...	Language needed	What you do
“What is the probability / chance this loan goes bad?”	Probability	Full Q1 recipe: plug values → z → sigmoid → p → compare to threshold
“The odds of default are 3-to-1 — what's the probability?”	Odds → Probability	3÷(3+1) = 75%. One ladder move, no model needed
“Interpret the coefficient 0.8 on PastDefault”	Odds (via odds ratio)	$e^{0.8} \approx 2.2$ → 'a past default multiplies the ODDS of default by ≈2.2'
“The model output shows z = 1.6 for this customer”	Log-odds → Probability	Squash it: σ(1.6) ≈ 0.83 → an 83% chance

THE SENTENCE THAT CATCHES EVERYONE

“Odds of 4” is NOT a probability of 4 (impossible) and NOT 40%. It is p = 4÷5 = 80%. Whenever a number bigger than 1 appears as a 'chance', it must be odds — translate before you interpret.

So how was Q1's z calculation different from Q2?

They use the same equation but from opposite ends — one **pulls the lever**, the other **describes the lever**:

	Q1 — “What's the probability?”	Q2 — “Interpret this coefficient”
You are given	The equation AND one specific customer's values (DTI = 40, PastDefault = 1)	Only the equation — no customer at all
What you compute	z for that person (ALL coefficients at once), then sigmoid → p, then a decision	e^b for ONE coefficient — the odds ratio. No z, no sigmoid, no decision
The answer looks like	“p ≈ 0.40, above the 0.30 cut-off → reject”	“A past default multiplies default odds by ≈2.2, other things equal”
Cooking analogy	Using the whole recipe to cook one dish	Explaining what one ingredient does to the flavour

THE DEMONSTRATION THAT WELDS Q1 AND Q2 TOGETHER (WORTH READING TWICE)

Take our Q1 customer (DTI 40, past default): z = −0.4, so odds = $e^{-0.4}$ = 0.67 and p ≈ 40%. Now imagine the SAME customer WITHOUT the past default: z = −3.2 + 0.05×40 = −1.2, odds = $e^{-1.2}$ = 0.30, p ≈ 23%.

Compare the two versions of this customer: odds went 0.30 → 0.67. Divide them: 0.67 ÷ 0.30 = **2.23 = $e^{0.8}$ exactly.** That is what 'the 0.8 on PastDefault' means — flip that switch and the odds multiply by 2.2, for ANY customer.

But look at the probabilities: 23% → 40%, a jump of 17 points. Now do it for a customer who was already very risky, say z = +2.0 (p = 88%): adding the past default gives z = 2.8, p = 94% — a jump of only 6 points. **Same coefficient, same 2.2× on the odds, different jump in probability.** This is the 'swing' from your logistic guide, now in numbers: the odds ratio is the coefficient's FIXED promise; the probability effect depends on where the customer already sits on the S-curve. This is exactly why Q2 answers are phrased in odds, never in 'adds X% to the probability'.

EXAM HOOK

Coefficient → adds to the log-odds (constant). e^b → multiplies the odds (constant). Effect on the probability → NOT constant (big near 50%, tiny near the ends). Three scales, three behaviours — a favourite true/false hunting ground.

2. The confusion matrix, actually drawn

The story: a bank tests its default model on **200 past applicants**. In reality **40 defaulted** and 160 repaid. The model flagged **50** applicants as 'will default'; of those flagged, **30 really did default**. Positive class = 'will default'.

Filling the grid — always in this order: TP → FP → FN → TN

(1) TP is stated: 30 flagged-and-defaulted. **(2)** FP from the flagged total: 50 flagged − 30 correct = 20. **(3)** FN from the actual total: 40 real defaulters − 30 caught = 10. **(4)** TN is everything left: 200 − 30 − 20 − 10 = 140. Cross-check: 30+10 = 40 ✓ and 30+20 = 50 ✓.

	ACTUALLY defaulted	ACTUALLY repaid	Row total
Model says DEFAULT	**TP = 30** caught ✓ *correctly rejected*	**FP = 20** false alarm ✗ (Type I) *good customer turned away*	50 flagged
Model says REPAY	**FN = 10** missed ✗ (Type II) *loan given, money lost*	**TN = 140** cleared ✓ *good loan made*	150
Column total	40 real defaulters	160 real repayers	**200**

HOW TO READ THE GRID WITHOUT EVER MIXING IT UP

The **diagonal (green)** is where the model and reality agree — TP and TN, the correct calls. The **off-diagonal (red)** is where they disagree — the two kinds of mistake. Rows = what the MODEL said; columns = what REALITY was. Every metric is just a ratio of cells:

- **Accuracy** = green ÷ everything = (30+140)÷200 = **85%** — 'right about 85 of every 100 applicants.'
- **Precision** = TP ÷ its ROW = 30÷50 = **60%** — 'of the 50 people we flagged, only 3 in 5 were real defaulters; 2 in 5 were good customers we scared away.' Precision lives in the model's row because it judges the model's CLAIMS.
- **Recall / Sensitivity** = TP ÷ its COLUMN = 30÷40 = **75%** — 'of the 40 genuine defaulters, we caught 30 and let 10 through.' Recall lives in reality's column because it judges coverage of the REAL cases. (Sensitivity is the same number — the medical name.)
- **Specificity** = TN ÷ the other column = 140÷160 = **87.5%** — 'of the genuinely good customers, we correctly cleared 7 in 8.'
- **F1** = 2×(0.60×0.75)÷(0.60+0.75) = 0.90÷1.35 = **0.67** — the balanced grade, dragged toward the weaker of precision and recall.

EXAM HOOK — THE CELL-TO-COST DICTIONARY (THIS IS WHAT MAKES YOU A MANAGER)

Each red cell has a price. Here: FN = a loan disbursed to a defaulter → lose the principal (huge). FP = a creditworthy customer rejected → lose the interest margin (small). The moment you can price the two red cells, every later question — metric choice, model choice, threshold — becomes arithmetic. Hold onto these 200 applicants; the next section reuses them.

3 & 4. “Why is this model better?” — the same bank, a second model, and money

You said the two-model comparison felt like 'just numbers'. Fair — because percentages float free until each error carries a price. So let's attach prices: this bank loses on average **₹5,00,000** when a defaulter slips through (FN), and forgoes about **₹50,000** of interest profit when it wrongly rejects a good customer (FP). Correct cells cost nothing extra — they are the model doing its job.

Enter Model B — tested on the SAME 200 applicants

Model B is far more aggressive: it flags 95 applicants and catches 38 of the 40 real defaulters. Its matrix: TP = 38, FP = 57, FN = 2, TN = 103.

	Model A (from Section 2)	Model B (aggressive)	Who wins this line?
Accuracy	(30+140)÷200 = **85%**	(38+103)÷200 = **70.5%**	A — by 14.5 points!
Precision	30÷50 = 60%	38÷95 = 40%	A
Recall (catch rate on defaulters)	30÷40 = 75%	38÷40 = **95%**	B — misses only 2, not 10
F1	0.67	2×(0.40×0.95)÷1.35 = 0.56	A
Misses (FN)	10 defaulters approved	2 defaulters approved	B
False alarms (FP)	20 good customers rejected	57 good customers rejected	A

MY THINKING — WATCH THE DECISION GET MADE

On the scoreboard, A wins 4 lines out of 6, including the headline accuracy. A lazy answer picks A. Now watch what happens when we price the red cells instead of counting green ones:

Model A's error bill: 10 misses × ₹5,00,000 = ₹50,00,000, plus 20 false alarms × ₹50,000 = ₹10,00,000 → **total ₹60 lakh** per 200 applications.

Model B's error bill: 2 misses × ₹5,00,000 = ₹10,00,000, plus 57 false alarms × ₹50,000 = ₹28,50,000 → **total ₹38.5 lakh**.

Model B saves ₹21.5 lakh per 200 applicants despite being 14.5 points 'less accurate'.

Why did accuracy lie? Because accuracy treats every applicant as equally important — it happily trades one caught defaulter (worth ₹5 lakh) for ten cleared repayers (worth ₹50k of avoided annoyance each... no, worth nothing extra at all). Accuracy counts heads; the business counts rupees. When errors have unequal prices, the metric that tracks the EXPENSIVE error (here recall, guarding the ₹5-lakh misses) is the one that predicts the money.

The exam sentence: “Although Model A has higher accuracy, the costlier error here is the miss; Model B's recall of 95% cuts misses from 10 to 2, reducing total error cost from ₹60L to ₹38.5L — deploy B, and manage its false alarms through review capacity or a slightly higher threshold.”

The same routine where the verdict FLIPS — proof it's a method, not a slogan

SPAM FILTER: NOW THE FALSE ALARM IS THE EXPENSIVE CELL

1,000 emails, 200 are spam. Positive = spam. The costly error flips: an FP means a REAL email (perhaps a job offer) is binned — call it ₹5,000 of damage. An FN just means one spam reaches the inbox — ₹5 of annoyance.

Model S (careful): TP = 140, FP = 2, FN = 60 → precision 140÷142 ≈ 99%, recall 70%. **Model T (aggressive):** TP = 190, FP = 34, FN = 10 → precision 190÷224 ≈ 85%, recall 95%.

T looks heroic on recall — the exact quality that won the bank case. But bill the errors: **S:** 2×₹5,000 + 60×₹5 = ₹10,300. **T:** 34×₹5,000 + 10×₹5 = ₹1,70,050. **S wins by a factor of 16**, because here every false alarm is catastrophic and every miss is trivial.

Same five steps, opposite verdict. This is the whole lesson of Type 5: **never ask 'which model has better numbers?'; ask 'which model makes fewer of the mistakes this business cannot afford?'** The cost structure, not the scoreboard, picks the winner.

EXAM HOOK — IF NO RUPEE COSTS ARE GIVEN

State the cost logic in words instead: name the positive class, name both errors, say which is plausibly costlier and why, then pick the model stronger on that error's metric. Examiners award the reasoning chain; the rupees above are just that chain made visible.

5. Thresholds — a real calculation, not hit-and-trial

Your instinct was right to be suspicious: 'lower it to 0.3' sounds like guesswork. It isn't — here is the machinery the textbook hides. The model gives EVERY case a probability score. Sort the cases by score, and every distinct score becomes a candidate cut-line. There are only finitely many candidates, each one's confusion matrix can be counted exactly, and you pick the candidate that scores best on your chosen rule. That is a **systematic sweep with exact arithmetic at each stop** — exhaustive evaluation, not trial-and-error guessing.

Watch the sweep on ten real patients

Ten patients, scored by a disease model. D = truly diseased (4 of them), H = truly healthy (6). Sorted by score:

Score	0.92	0.85	0.74	0.66	0.58	0.45	0.33	0.21	0.15	0.08
Truth	D	D	H	D	H	D	H	H	H	H

A threshold is a vertical line dropped into this sorted row: everyone to its left is called Diseased, everyone to its right is called Healthy. Slide the line and count the cells at each stop:

Threshold	Called positive	TP	FP	FN	TN	Sensitivity (TP÷4)	Specificity (TN÷6)	Youden J = Sens+Spec−1	Cost = ₹5×FP + ₹100×FN
0.7	top 3	2	1	2	5	0.50	0.83	0.33	₹205
0.5	top 5	3	2	1	4	0.75	0.67	0.42	₹110
0.4	top 6	4	2	0	4	1.00	0.67	**0.67 ← max**	**₹10 ← min**
0.3	top 7	4	3	0	3	1.00	0.50	0.50	₹15

MY THINKING, NARRATED — READ THE TABLE LIKE THIS

Move the cut from 0.7 down to 0.4 and watch the mechanics: each step LEFTward converts some cases from 'called healthy' to 'called diseased'. If the converted case was truly diseased, an FN becomes a TP (recall rises — good). If it was truly healthy, a TN becomes an FP (false alarms rise — the price). That is the entire trade-off, made physical: **lowering the threshold buys catches and pays in false alarms.**

Now both selection rules are just column-reading: **Cost rule** — with a miss at ₹100 and a false alarm at ₹5, the cost column bottoms out at threshold 0.4 (₹10). **Youden rule** — the J column peaks at the same 0.4 (0.67). Two different philosophies, one sweep, and in this dataset they even agree (they don't always: extreme costs drag the costed answer further than the balanced one).

In the exam you will never sweep fifty candidates — you'll be handed two or three thresholds with their FP/FN (or Sens/Spec) counts, exactly like this table's rows. Your job is only to compute the last column(s) and crown the winner.

Youden's J, finally in plain English

WHAT J ACTUALLY MEASURES — AND WHAT THAT 0.55 WAS

Sensitivity = share of real Yeses you catch. Specificity = share of real Nos you clear. A model with ZERO skill can still make Sens + Spec = 1 at every threshold — flag everyone (Sens 1, Spec 0), flag no one (Sens 0, Spec 1), or any coin-flip in between. That skill-free trade line IS the ROC diagonal.

So J = Sens + Spec − 1 measures **how far above that no-skill line your threshold sits** — the pure skill at that cut, on a 0-to-1 scale. J = 0: no better than guessing. J = 1: perfect (catch everything AND clear everything). Geometrically, J is the vertical height of the ROC point above the diagonal, which is why 'maximise J' = 'pick the ROC point that bulges highest above the diagonal'.

Now the 0.55 confusion, cleared: in the playbook example, some candidate cut-off (say 0.4) delivered Sensitivity 0.85 and Specificity 0.70. Its J = 0.85 + 0.70 − 1 = 0.55. **That 0.55 is the cut-off's SCORE — its marks out of 1 — not a threshold.** You compute a J score for every candidate threshold and pick the threshold whose J is highest. Two different numbers live in this method: the threshold (a probability cut like 0.4) and its J (a quality grade like 0.55). Never let them swap places.

EXAM HOOK — WHICH SELECTION RULE WHEN

Costs given or clearly implied (“a miss costs 20× a false alarm”) → minimise C1×FP + C2×FN. No costs, question says 'balanced' or 'best overall cut-off' → maximise Youden's J. Either way, moving the threshold re-slices the SAME model — its AUC does not change; only the confusion matrix does. (Bonus name-drop: building the cost asymmetry into training itself is Cost-Sensitive Logistic Regression, CS-LR — it deliberately trades a sliver of accuracy for better recall/F1.)

6. Gap-check — what your three session guides cover that the playbook skipped

I re-read all three uploaded guides against the playbook. These eight items are interpretation-testable and weren't given their own space. Each is compressed to its exam essence.

6.1 Decision boundary vs threshold (logistic guide, Section 6)

Two different dials that move together. The **threshold** lives on the CHANCE side ('flag if p ≥ 65%'); the **decision boundary** is the matching flip-point on the INPUT side ('rating 3.2'). Course example: an employee is promoted if rating exceeds 3.2, and at exactly 3.2 the model's chance is 65% — so 3.2 is the boundary, 65% is the threshold. Exam trap: they'll give you both numbers and ask which is which. Input-units number = boundary; percentage = threshold.

6.2 The model health-checks table (logistic guide, Section 5)

Check	The one-line interpretation you write
Wald's test	“Does THIS ONE input matter?” — logistic's cousin of the t-test; $p < 0.05$ → the variable earns its place
Likelihood Ratio (LR) test	“Does the WHOLE model beat a do-nothing guess?” — cousin of the F-test; $p < 0.05$ → model is worth having
Pseudo R^2	Rough 'how good is the fit' for logistic (R^2's cousin) — compare models with it, don't worship its absolute value
AIC / BIC	Model-comparison scores: reward fit, punish complexity; LOWER = better; BIC punishes complexity harder

6.3 Multi-class results: macro vs micro vs weighted (decision-tree guide, Part B)

With 3+ classes, collapse to 2×2 one class at a time (**one-vs-all**): that class's TP is its diagonal cell; FN = rest of its actual column; FP = rest of its predicted row; TN = everything else. Then to average per-class scores:

Average	How	When it's the right one
Macro	Simple mean of per-class scores — every class equal	When small classes matter as much as big ones (rare product categories)
Micro	Pool ALL TP/FP/FN globally first, then compute	When big classes should dominate. KEY IDENTITY: in single-label problems micro precision = micro recall = micro F1 = ACCURACY — a classic exam line
Weighted	Mean weighted by class size (support)	The compromise most software reports by default

6.4 Regression trees (decision-tree guide, A.9)

When the tree predicts a NUMBER, impurity is replaced by spread: candidate splits are compared on **SSE** (sum of squared errors) and the **lowest SSE wins** (course case: candidates 172.67 / 134.5 / 50 → split at the 50). A leaf's prediction = the plain **average of the training cases in that leaf**. Interpretation question style: “three candidate splits have SSE 90, 60, 75 — which does the tree choose and what will the left leaf predict?” → the 60, and the mean of its members.

6.5 Reading residual plots (linear-regression guide, 4.6)

You are shown	Diagnosis	Fix to prescribe
Residuals-vs-fitted plot shaped like a funnel/cone	Heteroscedasticity — spread grows with the prediction; results imprecise, confidence overstated	Weighted least squares
A visible curve/pattern in the residuals	The model missed a real (non-linear) pattern	Transform a variable (e.g. take logs) or use a non-linear model — course case: log transform lifted R^2 57% → 99%
PP plot: points on a straight diagonal	Residuals normal — all good	Nothing; say 'assumption satisfied'
PP plot: S-shape	Normality violated	Transform y, or suspect a missing non-linearity

6.6 Outliers: leverage vs influence, and Cook's distance (linear guide, 4.7)

Leverage point = extreme on the inputs but ON the trend → has the POTENTIAL to move the line, doesn't. **Influential** point = actually DRAGS the line and changes results. The direct influence meter is **Cook's distance: greater than 1 = an influential outlier** — the classic exam line. (Mahalanobis flags 'far from the data's centre'; leverage values flag extreme inputs.)

6.7 Underfitting — the other half of the train/test story (linear guide, 5.2)

The playbook diagnosed overfitting (great on train, poor on test = high variance → prune/ensemble). The mirror case: **poor even on TRAINING data = high bias = underfitting** — the model is too simple (a straight line forced through a curve). Fix: more complexity/features, not less. Exams test BOTH directions: read where the failure shows up. Train bad → bias. Train great, test bad → variance.

6.8 Two spare one-liners worth carrying in

• **Dummy coefficient in LINEAR regression:** intercept = the baseline group's average; b = the average DIFFERENCE between groups (significant if $p < 0.05$). (In logistic, the same dummy speaks in odds: e^b × the odds.)
• **AUC's plain meaning** (worth quoting verbatim): the probability the model ranks a randomly chosen real-Yes above a randomly chosen real-No — which is why 0.5 is a coin flip.

Mini-drill on exactly these five topics

Cover the right column. Every answer is one ladder move or one table-read away.

Question	Answer + thinking
1. p = 0.25. Odds?	0.25÷0.75 = 1÷3 ≈ 0.33 — one default for every three repayments.
2. Odds = 9. Probability?	9÷(9+1) = 90%. (Odds > 1 always means p > 50%.)
3. Coefficient on 'owns home' = −0.7 in a default model. Interpret.	$e^{-0.7} \approx 0.50$: homeowners have about HALF the odds of default, other things equal. No z, no sigmoid — no customer was given.
4. 300 claims, 60 fraudulent. Model flags 90; 48 truly fraudulent. Draw the cells.	TP 48, FP 42, FN 12, TN 198. Precision 48÷90 = 53%, Recall 48÷60 = 80%, Accuracy 246÷300 = 82%.
5. In Q4's insurer, a paid-out fraud costs ₹2L, an investigation of a clean claim ₹10k. Error bill?	12×2,00,000 + 42×10,000 = ₹24L + ₹4.2L = ₹28.2L. A rival model with recall 90% (FN 6) and FP 70 bills 6×2L + 70×0.1L = ₹19L → deploy the rival despite more false alarms.
6. Candidates: cut 0.5 (Sens .70, Spec .80) vs cut 0.35 (Sens .88, Spec .64). Balanced choice?	J(0.5) = 0.50; J(0.35) = 0.52 → pick threshold 0.35. The 0.52 is its SCORE, not a threshold.

THE ONE-PARAGRAPH VERSION OF THIS WHOLE SUPPLEMENT

Probability, odds and log-odds are three dialects of one belief — the model computes in log-odds, coefficients speak in odds (e^b), managers decide in probability. Q1 pulls the whole lever for one customer; Q2 describes one lever in general. The confusion matrix is two green cells of agreement and two red cells of mistake — and the moment you price the red cells, 'which metric', 'which model' and 'which threshold' all become the same calculation: minimise the rupee value of the mistakes. Thresholds are chosen by an exact sweep over sorted scores — costed (min C1×FP + C2×FN) when prices exist, balanced (max Youden's J = Sens + Spec − 1, a SCORE for each candidate cut) when they don't.

End of supplement · Machine Learning for Managers · Kishore Kunal (38BM21)